

Course: Networked Control

Computer session 01: Stability and simulation of the
solutions of linear systems and LMIs

Prof. Marcello Farina

Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)

Politecnico di Milano, Italy

`marcello.farina@polimi.it`

Outline

- Stability and simulation of the solutions of linear systems
- Linear matrix inequalities
- Exercizes

Outline

- **Stability and simulation of the solutions of linear systems**
- Linear matrix inequalities
- Exercizes

Continuous-time systems

Continuous-time linear time-invariant systems are defined by the quadruple (A, B, C, D) , e.g., with function 'ss'

`ss` Construct state-space model or convert model to state space.

Construction:

systemCT = **ss(A,B,C,D)** creates an object **SYS** representing the continuous-time state-space model

$$\frac{dx}{dt} = Ax(t) + Bu(t)$$

$$y(t) = Cx(t) + Du(t)$$

You can set $D=0$ or $D=[]$ to define the zero matrix of appropriate size.

```
>> A=[-1 0;0.2 -0.2]; B=[1 0.1]'; C=[0 1]; D=0;
```

Continuous-time systems

```
>> systemCT=ss(A,B,C,D)
```

```
systemCT =
```

```
a =
```

	x1	x2
x1	-1	0
x2	0.2	-0.2

```
b =
```

	u1
x1	1
x2	0.1

```
c =
```

	x1	x2
y1	0	1

```
d =
```

	u1
y1	0

Continuous-time state-space model.

Continuous-time systems

System responses can be obtained using, e.g.

```
[Y,T] = step(systemCT); % forced response to stepwise inputs  
[Y,T] = impulse(systemCT); % impulse response  
Y = lsim(systemCT,U,T); % forced response to input signal U
```

Otherwise, the explicit expressions can be used, e.g.,

For a time instant t

```
Xfree=expm(A*t)*x0; % free motion starting from initial  
                    condition x0. Warning: to compute the  
                    matrix exponential 'expm' must be  
                    used!! The function 'exp', if applied  
                    to a matrix, computes its element-wise  
                    exponential!!
```

If A is invertible and with constant input u :

```
Xforced=-A\ (eye(n)-expm(A*t)) *B*u;
```

Continuous-time systems

The stability properties can be analyzed studying the eigenvalues of matrix A , i.e.,

```
>> eig(A)
>> rho=max(real(eig(A))) % this is the spectral abscissa,
                           i.e., the real value of the
                           eigenvalue with maximum real
                           value
>> [V,D] = eig(A) % produces a diagonal matrix D of
                   eigenvalues and a full matrix V whose
                   columns are the corresponding
                   eigenvectors so that  $A*V = V*D$ . Warning:
                   V is not invertible when A is not
                   diagonalizable!!
>> [V,J] = jordan(A) % computes the Jordan Canonical/Normal
                      Form of the matrix A and the
                      similarity transformation, V, so
                      that  $V\backslash A*V = J$ . The columns of V
                      are the generalized eigenvectors. Of
                      course, if A is diagonalizable D is
                      diagonal!
```

Continuous-time systems

```
>> poly(A) % when A is an n by n matrix, is a row vector  
           with n+1 elements which are the coefficients of  
           the characteristic polynomial,  
            $\det(\lambda \text{eye}(\text{size}(A)) - A)$ .  
  
>> roots(poly(A)) % roots of the characteristic polynomial,  
                   i.e., the eigenvalues
```


Discretization

c2d Converts the continuous-time dynamic system

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx + Du\end{aligned}$$

to discrete time, i.e.,

$$\begin{aligned}x_{k+1} &= Fx_k + Gu_k \\ y_k &= Hx_k + Lu_k\end{aligned}$$

>> **systemDT = c2d(systemCT,TS,METHOD)** computes a discrete-time model systemDT with sample time TS that corresponds to (or approximates) the continuous-time model systemCT. The string METHOD selects the discretization method among the following:

'zoh'	Zero-order hold on the inputs
'foh'	Linear interpolation of inputs
'impulse'	Impulse-invariant discretization
'tustin'	Bilinear (Tustin) approximation.
'matched'	Matched pole-zero method (for SISO systems only).

The default is 'zoh' when METHOD is omitted. The sample time TS should be specified in the time units of SYSC.

Given systemDT, the corresponding system matrices can be obtained as follows

```
>> [F,G,H,L,TS] = ssdata(systemDT) % ssdata can be used also  
                                     for continuous-time  
                                     systems (in such case Ts=0)
```

Discretization

```
>> systemDT=c2d(systemCT,1)
systemDT =
```

```
a =
```

	x1	x2
x1	0.3679	0
x2	0.1127	0.8187

```
b =
```

	u1
x1	0.6321
x2	0.1592

```
c =
```

	x1	x2
y1	0	1

```
d =
```

	u1
y1	0

Sample time: 1 seconds

Discrete-time state-space model.

Discretization

```
>> [F,G,H,L,Ts]=ssdata(systemDT)
```

F =

```
0.3679    0
0.1127    0.8187
```

G =

```
0.6321
0.1592
```

H =

```
0    1
```

L =

```
0
```

Ts =

```
1
```

Discrete-time systems

Discrete-time linear time-invariant systems can be defined by the quadruple (F, G, H, L) and the sample time T_s with function 'ss'

```
>> F=[0.3679 0;0.1127 0.8187]; G=[0.6321 0.1592]'; H=[0 1];  
    L=0; Ts=1;  
>> systemDT = ss(F,G,H,L,Ts)  
systemDT =  
    a =  
           x1      x2  
    x1    0.3679      0  
    x2    0.1127    0.8187  
    b =  
           u1  
    x1    0.6321  
    x2    0.1592  
    c =  
           x1      x2  
    y1      0      1  
    d =  
           u1  
    y1      0  
Sample time: 1 seconds
```

Discrete-time systems

System responses can be obtained using, e.g.

```
[Y,T] = step(systemTD); % forced response to stepwise inputs  
[Y,T] = impulse(systemTD); % impulse response  
Y = lsim(systemTD,U,T); % forced response to input signal U
```

Otherwise, the explicit expressions can be used, e.g.,

Fix a time step k

```
Xfree=(F)^k*x0; % free motion starting from initial  
                condition x0
```

If $(\text{eye}(n)-F)$ is invertible and with constant input u :

```
Xforced=(eye(n)-F)\(eye(I)-F^k)*G*u;
```

Discrete-time systems

The stability properties can be analyzed studying the eigenvalues of matrix F , i.e.,

```
>> eig(F)
>> rho=max(abs(eig(F))) % this is the spectral radius,
                        i.e., the absolute value of the
                        eigenvalue with maximum absolute
                        value
>> [V,D] = eig(F) % produces a diagonal matrix D of
                  eigenvalues and a full matrix V whose
                  columns are the corresponding
                  eigenvectors so that  $F*V = V*D$ . Warning:
V is not invertible when F is not
diagonalizable!!
>> [V,J] = jordan(F) % computes the Jordan Canonical/Normal
                     Form of the matrix F and the
                     similarity transformation, V, so
                     that  $V^{-1}F*V = J$ . The columns of V
                     are the generalized eigenvectors. Of
course, if F is diagonalizable D is
diagonal!
```

Discrete-time systems

```
>> poly(F) % when A is an n by n matrix, is a row vector  
            with n+1 elements which are the coefficients of  
            the characteristic polynomial,  
             $\det(\lambda \text{eye}(\text{size}(A)) - A)$ .  
  
>> roots(poly(F)) % roots of the characteristic polynomial,  
                    i.e., the eigenvalues
```

Outline

- Stability and simulation of the solutions of linear systems
- **Linear matrix inequalities**
- Exercizes

Linear matrix inequalities (LMIs)

Inequalities of type

$$A^T P + P A < 0$$

$$F^T P F - P < 0$$

$$P > 0$$

are LMIs.

There are various toolboxes that allow to solve LMI optimization problems, i.e., optimization problems where LMIs are enforced, e.g.,

- YALMIP
- CVX
- ...

In this course we learn how to use YALMIP (at least to solve LMIs of our interest).

Linear matrix inequalities (LMIs)

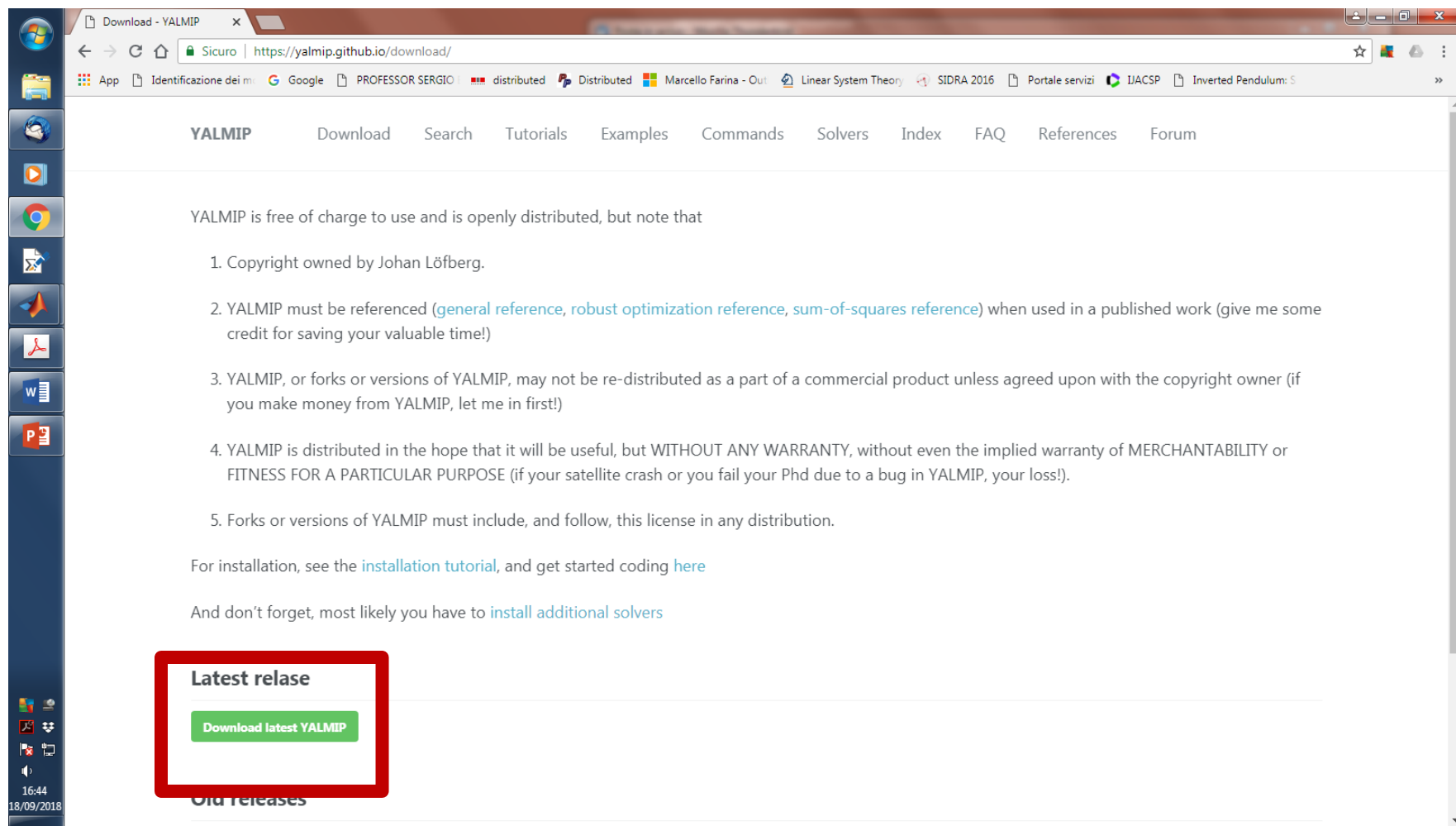
1. Download YALMIP at <https://yalmip.github.io/>

The screenshot shows a web browser window with the YALMIP website. The address bar displays <https://yalmip.github.io/>. The website's navigation bar includes links for YALMIP, Download, Search, Tutorials, Examples, Commands, and Solvers. A large banner image of a forest features the YALMIP logo and a 'Download' button, which is highlighted with a red rectangle. Below the banner, the website lists several featured articles, each with a title, an update date, and a brief description.

Article Title	Update Date	Description
Equalities with uncertainty	Updated: September 11, 2018	$\neq =$, Horse purchases and warehouse logistics
New release R20180817	Updated: August 17, 2018	Minor fixes and improvements
New release R20180612	Updated: June 12, 2018	Minor fixes and improvements
Parameterizing the uncertainty set in robust optimization	Updated: April 25, 2018	Uncertainty descriptions can only involve uncertain variables, so how can they be parameterized?
New release R20180413	Updated: April 13, 2018	Minor fixes and improvements
Designing polynomials	Updated: February 16, 2018	Working with polynomials, function values, derivatives, integrals and their properties
New release R20180209	Updated: February 09, 2018	Minor fixes and improvements
Multi-objective problems in YALMIP	Updated: November 24, 2017	
New release R20171121	Updated: November 21, 2017	Minor fixes and improvements
New release R20171018	Updated: October 18, 2017	Important patch
Modelling if-else-end statements	Updated: October 17, 2017	

Linear matrix inequalities (LMIs)

1. Download YALMIP at <https://yalmip.github.io/>



The screenshot shows a web browser window with the URL <https://yalmip.github.io/download/>. The page has a navigation bar with links: YALMIP, Download, Search, Tutorials, Examples, Commands, Solvers, Index, FAQ, References, and Forum. The main content area states that YALMIP is free of charge and openly distributed, but notes that it is copyright owned by Johan Löfberg. It lists five conditions for use and distribution, including referencing YALMIP in published work and not re-distributing it as part of a commercial product. It also provides links to the installation tutorial and additional solvers. At the bottom, there is a section for the latest release, which is highlighted with a red box, and a green button labeled "Download latest YALMIP".

Download - YALMIP x

Sicuro | <https://yalmip.github.io/download/>

App | Identificazione dei m... | Google | PROFESSOR SERGIO | distributed | Distributed | Marcello Farina - Out | Linear System Theory | SIDRA 2016 | Portale servizi | IJACSP | Inverted Pendulum: S

YALMIP Download Search Tutorials Examples Commands Solvers Index FAQ References Forum

YALMIP is free of charge to use and is openly distributed, but note that

1. Copyright owned by Johan Löfberg.
2. YALMIP must be referenced ([general reference](#), [robust optimization reference](#), [sum-of-squares reference](#)) when used in a published work (give me some credit for saving your valuable time!)
3. YALMIP, or forks or versions of YALMIP, may not be re-distributed as a part of a commercial product unless agreed upon with the copyright owner (if you make money from YALMIP, let me in first!)
4. YALMIP is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY, without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE (if your satellite crash or you fail your Phd due to a bug in YALMIP, your loss!).
5. Forks or versions of YALMIP must include, and follow, this license in any distribution.

For installation, see the [installation tutorial](#), and get started coding [here](#)

And don't forget, most likely you have to [install additional solvers](#)

Latest release

[Download latest YALMIP](#)

Old releases

Linear matrix inequalities (LMIs)

2. Download sedumi solver at <https://github.com/SQLP/SeDuMi>

The screenshot shows the GitHub repository page for `sqlp / sedumi`. The repository has 20 watches, 101 stars, and 53 forks. The license is GPL-2.0. The repository contains 39 commits, 2 branches, 0 releases, and 3 contributors. The latest commit is by mcg1969, titled "Merge pull request #28 from sqlp/mac_recompile", dated 16 Dec 2017. The repository includes files such as `conversion`, `doc`, `examples`, `o_win`, and `.gitignore`. A red box highlights the "Clone or download" button and the "GPL-2.0" license information.

Linear matrix inequalities (LMIs)

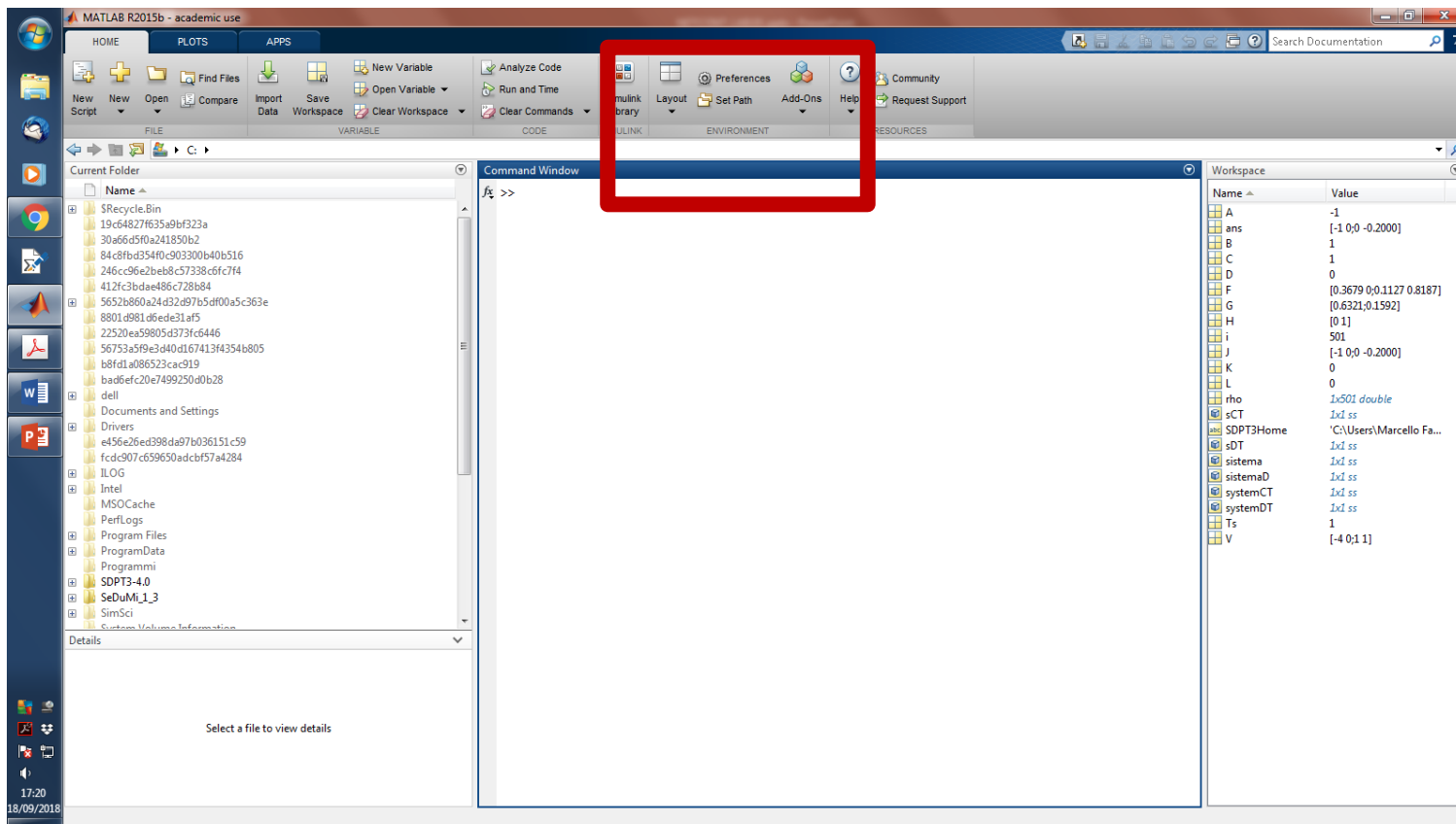
3. Download SDTP3 solver at <https://github.com/SQLP/SDPT3>

The screenshot shows the GitHub repository page for `sqlp / sdpt3`. The page includes a navigation bar with links to Features, Platform, Business, Explore, and Pricing. Below the repository name, there are statistics: 26 commits, 2 branches, 0 releases, and 2 contributors. A red box highlights the '2 contributors' information and the 'Clone or download' button. The page also features a 'Join GitHub today' banner and a list of files including Examples, HSDSolver, Solver, dimacs, and sdplib.

sdpt3-master.zip sdpt3-master.zip SeDuMi_1_3.zip YALMIP-master.zip

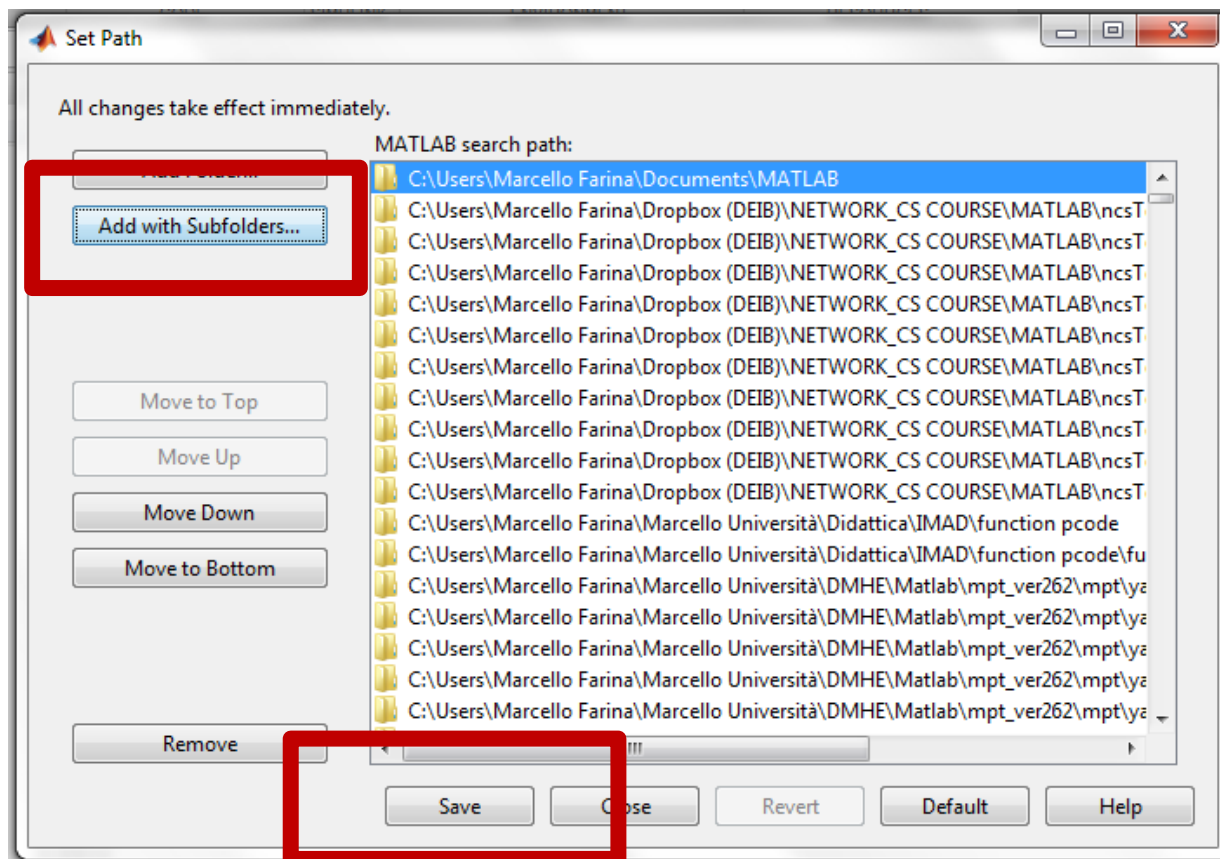
Linear matrix inequalities (LMIs)

4. Unzip Yalmip, sedumi, and SDTP3 folders in an appropriate position
5. Open Matlab
5. Add the three unzipped folders (with subfolders) to the Matlab path



Linear matrix inequalities (LMIs)

4. Unzip Yalmip, sedumi, and SDTP3 folders in appropriate positions
5. Open Matlab
6. Add the three unzipped folders (with subfolders) to the Matlab path



Linear matrix inequalities (LMIs)

1. Clear Yalmip memory, before to start a new LMI problem

```
>> yalmip clear
```

2. Create symbolic decision variables

You can create a `sdpvar` variable by:

<code>P = sdpvar(n)</code>	Symmetric nxn matrix
<code>P = sdpvar(n,n)</code>	Symmetric nxn matrix
<code>P = sdpvar(n,m)</code>	Full nxm matrix (n~=m)

Definition of multiple scalars can be simplified

```
sdpvar x y z w
```

The parametrizations supported are

<code>P = sdpvar(n,n,'full')</code>	Full nxn matrix
<code>P = sdpvar(n,n,'symmetric')</code>	Symmetric nxn matrix

Linear matrix inequalities (LMIs)

3. Define linear inequality constraints

```
(1) L=[P>0]; % A should be numerical and defined  
L=L+[A'*P+P*A<0] beforehand
```

```
(2) L=[P>k*eye(n)] % k>0 can be either a  
L=L+[A'*P+P*A<-k*eye(n)] variable to be defined  
with sdpvar) or a constant
```

➡ It may be necessary to replace (1) with (2) if the solver does not support strict inequalities, i.e., interprets ' $<$ ' as ' $\leq$ '.

4. Define a cost function to be minimized (optional), linear in the symbolic decision variable, e.g.,

```
J=trace(P)
```

5. Solution to the LMI optimization (or feasibility) problem

```
optimize(L,J)  
P=double(P); % assigns the numerical value to P
```

Outline

- Stability and simulation of the solutions of linear systems
- Linear matrix inequalities
- **Exercizes**

Exercise 1

Consider the continuous-time system

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx\end{aligned}$$

where

$$A = \begin{bmatrix} -1 & 0 \\ 0.2 & -0.2 \end{bmatrix}, B = \begin{bmatrix} 1 \\ 0.1 \end{bmatrix}, C = [0 \ 1]$$

- A) Compute the eigenvalues and the spectral abscissa of the system. Is the system open-loop asymptotically stable?
- B) Plot the free, forced, and total motions of the output of the system starting from the initial condition $x(0) = [1, 0]^T$ and with input $u(t) = 1$ for all $t \geq 0$.
- C) Compute the equilibrium point of the system when $u(t) = 1$ for all $t \geq 0$.
- D) Set up and solve an LMI feasibility problem for checking the stability of the open-loop dynamics.

Exercise 2

Define the discrete-time system

$$\begin{aligned}x_{k+1} &= Fx_k + Gu_k \\ y_k &= Hx_k\end{aligned}$$

by discretizing the continuous-time system in Exercise 1 with sampling time $h = 1$ s.

- A) Compute the eigenvalues and the spectral radius of the system. Is the system open-loop asymptotically stable?
- B) Plot the free, forced, and total motions of the output of the system starting from the initial condition $x_0 = [1, 0]^T$ and with input $u_k = 1$ for all $k \in \mathbb{N}$.
- C) Compute the equilibrium point of the system when $u_k = 1$ for all $k \in \mathbb{N}$.
- D) Set up and solve an LMI feasibility problem for checking the stability of the open-loop dynamics.

Exercise 3

Consider the continuous-time system

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx\end{aligned}$$

where

$$A = \begin{bmatrix} 0 & 0 \\ 0 & -1 \end{bmatrix}, B = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, C = [1 \ 1]$$

- A) Compute the eigenvalues and the spectral abscissa of the system. Is the system open-loop asymptotically stable?
- B) Plot the free, forced, and total motions of the output of the system starting from the initial condition $x(0) = [1, 0]^T$ and with input $u(t) = 1$ for all $t \geq 0$.
- C) Set up and solve an LMI feasibility problem for testing the stability of the open-loop dynamics.

Exercise 4

Define the discrete-time system

$$\begin{aligned}x_{k+1} &= Fx_k + Gu_k \\ y_k &= Hx_k\end{aligned}$$

by discretizing the continuous-time system in Exercise 3 with sampling time $h = 1$.

- A) Compute the eigenvalues and the spectral radius of the system. Is the system open-loop asymptotically stable?
- B) Plot the free, forced, and total motions of the output of the system starting from the initial condition $x_0 = [1, 0]^T$ and with input $u_k = 1$ for all $k \in \mathbb{N}$.
- C) Set up and solve an LMI feasibility problem for testing the stability of the open-loop dynamics.